

# Q-Calculus

Mathematical and Analyst Manual

Typed phonological conservativity under declared transformations

Exact finite inference, certificates, and principled refusal

Alexandre Menezes Barroso

[alexandrebarroso.com](http://alexandrebarroso.com)

Package version 2.0.0 · Protocol version 1.0.0 · 2026

Companion manual for Project PhonoScript

## Contents

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>1 Purpose, identity, and limits</b>                  | <b>1</b>  |
| 1.1 Three evidence layers . . . . .                     | 1         |
| <b>2 The prospective contract</b>                       | <b>1</b>  |
| <b>3 Answer types and exact values</b>                  | <b>2</b>  |
| <b>4 Formation, admission, and outcomes</b>             | <b>3</b>  |
| <b>5 The operational language</b>                       | <b>4</b>  |
| <b>6 Grouping, kernels, and quotient carriers</b>       | <b>5</b>  |
| 6.1 Minimal transversal enumeration . . . . .           | 5         |
| <b>7 Selected fibres, evidence cores, and decisions</b> | <b>5</b>  |
| <b>8 Query price and the information lattice</b>        | <b>6</b>  |
| <b>9 Contextual adequacy</b>                            | <b>7</b>  |
| <b>10 Recovery and first irreversible loss</b>          | <b>7</b>  |
| <b>11 Transports, bridges, and comparison</b>           | <b>8</b>  |
| <b>12 Serial semantics</b>                              | <b>8</b>  |
| <b>13 Finite inference and experiment design</b>        | <b>9</b>  |
| <b>14 Grammar adapters and ranking spaces</b>           | <b>9</b>  |
| 14.1 Query-relative ranking-space compression . . . . . | 10        |
| <b>15 Using Q-Calculus</b>                              | <b>11</b> |
| 15.1 Installation and local command line . . . . .      | 11        |
| 15.2 A ranking-space study . . . . .                    | 11        |
| 15.3 Comparing aligned ranking spaces . . . . .         | 12        |
| 15.4 A finite cross-formalism study . . . . .           | 12        |
| 15.5 Full JSON protocol . . . . .                       | 12        |
| 15.6 API and PhonoScript surfaces . . . . .             | 13        |
| <b>16 Certificates, canonical identity, and trust</b>   | <b>13</b> |
| <b>17 Proof ledger for QC-T01 through QC-T26</b>        | <b>14</b> |

|           |                                                          |           |
|-----------|----------------------------------------------------------|-----------|
| 17.1      | Formation and structural theorems . . . . .              | 14        |
| 17.2      | Context, recovery, and comparison theorems . . . . .     | 15        |
| 17.3      | Inference, planning, and verification theorems . . . . . | 16        |
| <b>18</b> | <b>Reproducibility, performance, and security</b>        | <b>17</b> |
| <b>19</b> | <b>Scientific use and claim ceilings</b>                 | <b>17</b> |
| <b>A</b>  | <b>Request reference</b>                                 | <b>18</b> |
| <b>B</b>  | <b>Refusal and blocker codes</b>                         | <b>18</b> |
| <b>C</b>  | <b>Paper-and-pencil procedure</b>                        | <b>19</b> |

## 1 Purpose, identity, and limits

Q-Calculus is a finite operational calculus for prospective, typed, query-relative preservation in phonological analysis. It asks whether a declared consequence survives a declared transformation. It does not ask whether two analyses are globally or metaphysically identical.

Let  $A_s$  and  $A_t$  be admitted source and target analyses, let  $Q$  be fixed before target evaluation, let  $R$  be the complete family of matched rows, and let  $\theta_Q$  be a typed answer transport. Directional conservativity is the condition

$$\theta_Q(\text{Ans}_{A_s}(Q, r)) = \text{Ans}_{A_t}(Q, r) \quad \text{for every } r \in R. \quad (1)$$

The equality in (1) is the semantic equality procedure of the target answer type. It is not equality of printed glyphs, rounded decimals, or chosen display fields.

The primitive is directional because a transformation may forget information. Symmetric equivalence is derived only when the analyses share a suitable answer bearer, or when verified transports and preservation judgments exist in both relevant directions. Thus “equivalence” is the appropriate symmetric relation derived from consequence-relative directional preservation.

The operational classifier is finite and resource bounded. An infinite or continuous result can enter through a registered theorem card or a proof-carrying finite response code, but that does not turn the finite engine into an unrestricted real-arithmetic or continuum decision procedure. The calculus also does not select the scientifically important query, infer a complete candidate set, convert a model consequence into an observation, or identify a mental representation from a minimum carrier.

### 1.1 Three evidence layers

This manual keeps three kinds of assurance separate.

1. The *mathematical specification* defines records, judgments, algorithms, and theorem statements.
2. The *implementation evidence* consists of exact execution, independent certificate checking, exhaustive and property tests, deterministic serialization, and bounded benchmarks.
3. The *scientific interpretation* depends on admitted sources, supports, bridges, empirical records, and claim ceilings.

The repository contains Lean declarations, a Wolfram package surface, and an operational Python implementation. Their existence is not itself a completed cross-engine audit. In the inspected checkout the Lean build log ends in a successful build, while the theorem registry still marks QC-T01 through QC-T26 as PENDING\_PROOF; the final axiom-audit, Wolfram, cross-engine, mutation, and release-marker reports are not present. This manual therefore distinguishes a stated theorem, a compiling formal projection, an executable path, and a promoted release result. None of those activities alone establishes source transcription, empirical adequacy, historical priority, or cognitive reality.

## 2 The prospective contract

**Definition 2.1** (Query profile). A query profile is a tuple

$$Q = (\lambda, U, t_Q, \tau_Q, A_Q, J_Q, \text{Nonterminal}_Q),$$

where  $\lambda$  is the scientific layer,  $U$  the bearer or domain,  $t_Q$  the readout term,  $\tau_Q$  the answer type,  $A_Q$  the evaluator and assumptions,  $J_Q$  the equality or judgment procedure, and  $\text{Nonterminal}_Q$  the policy for undefinedness, stopping, or nontermination when applicable.

The query is frozen prospectively. A user may ask only whether a winner is preserved, but may not change from complete-order preservation to winner preservation after seeing that loser order changed. A coarse query is legitimate; retrospective coarsening is not evidence for the original claim.

**Definition 2.2** (Q-record). A Q-record is an immutable, target-independent registration of the finite analysis space. It records the carrier, domains, scientific layers, questions, answer types, evaluators, supports, dependencies, architecture, contextual operations, transformations, transports, provenance, source digests, and resource policy. A target belongs to a request, not to the record.

Keeping targets outside the record has two consequences. First, one admitted record can support grouping, basis, selected-fibre, decision, evidence, and planning requests without changing its identity. Second, changing the target after formation produces a new request and fingerprint rather than silently changing the meaning of an old certificate.

The source and target branches of a comparison are evaluated independently:

$$\begin{array}{ccc} A_s & \xrightarrow{T} & A_t \\ \text{Ans}_{A_s}(Q) \downarrow & & \downarrow \text{Ans}_{A_t}(Q) \\ V_s(Q) & \xrightarrow{\theta_Q} & V_t(Q). \end{array}$$

The upper transformation  $T$  and lower transport  $\theta_Q$  are different objects. A candidate merger, evaluator change, or grammar-to-realization bridge does not by itself define how winners, probability laws, relations, traces, or intervals are compared.

### 3 Answer types and exact values

Every question names one answer type, and the public specification assigns an intended canonicalizer, verifier, and equality procedure to that type. The registry contains thirty-two identifiers. In the inspected Python tree, specialized canonicalization is implemented for a subset and the remaining branches use generic immutable canonical data; the class names printed in the registry are therefore not, by themselves, evidence of thirty-two distinct implemented verifier classes. The following grouping is a specification reference; the identifiers in monospace are the public names.

| Family                       | Registered answer types                                                                                                                            |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Logical and scalar           | BOOL, CATEGORICAL_CLOSED, CATEGORICAL_OPEN, INTEGER, RATIONAL, ALGEBRAIC_REAL, EXACT_SYMBOL                                                        |
| Finite structural            | FINITE_SET, FINITE_MULTISSET, TUPLE, OPTION, DEFINEDNESS_SUM, FINITE_RELATION, FINITE_PREORDER, FINITE_PARTITION                                   |
| Probability and serial       | FINITE_PROBABILITY_LAW, SYMBOLIC_PROBABILITY_LAW, TRACE_SET, STOPPED_LANGUAGE, STOPPED_PROBABILITY_LAW, WEIGHTED_PATH_SERIES, ALL_PREFIX_SEMANTICS |
| Symbolic algebra             | INTEGER_POLYNOMIAL, LAURENT_POLYNOMIAL, EXPONENTIAL_POLYNOMIAL, RATIONAL_FUNCTION, SEMIALGEBRAIC_REGION                                            |
| Intervals and theorem values | RATIONAL_INTERVAL, ALGEBRAIC_INTERVAL, PHASE_CELL, SYMBOLIC_THEOREM_VALUE, APPROXIMATE_INTERVAL                                                    |

A rational is canonical only when

$$b > 0, \quad \gcd(|a|, b) = 1.$$

An exact protocol value therefore uses signed decimal strings and an explicit positive denominator:

```
{"denominator":"2","kind":"rational","numerator":"1"}
```

Finite sets discard semantic duplicates and sort by canonical bytes; finite multisets retain positive multiplicities. That distinction is load-bearing for labelled MaxEnt candidates and path multiplicities. Finite probability laws require unique support keys, nonnegative rational masses, and

$$\sum_{x \in \text{supp } P} P(x) = 1.$$

DEFINEDNESS\_SUM distinguishes a defined value, semantic undefinedness, and registered divergence when the answer type licenses those branches. Semantic undefinedness is an evaluated value. NOT\_EVALUATED is an execution outcome saying that the requested relation never formed; they must not be identified.

The four serial carriers remain nominally different. A stopped language is a set of stopped readouts; a stopped probability law retains exact masses and nontermination mass; a weighted path series retains multiplicity and weights; all-prefix semantics retains every reachable prefix. No implicit cast exists between them.

Exact and approximate values are likewise disjoint. Overlapping approximate intervals do not prove equality of the unknown exact values, and a rounded decimal is never inserted into an exact rational, polynomial, or theorem path.

## 4 Formation, admission, and outcomes

Formation asks whether the syntactic and type-theoretic object exists. Admission asks whether the formed object licenses the requested scientific or mathematical relation. Evaluation computes only after both applicable gates pass.

The public execution statuses are EVALUATED, NOT\_EVALUATED, REFUSED, and INCONSISTENT. An evaluated comparison has exactly one of the verdicts

$$\text{CONSERVATIVE} \dot{\vee} \text{NONCONSERVATIVE}.$$

An unadmitted comparison produces NOT\_EVALUATED with blockers; this is not a third truth value between equality and inequality.

**Conservative.** Every registered matched row is independently evaluated, transported, and equal under target-type semantics.

**Nonconservative.** The same relation is admitted, but at least one exact mismatch exists. The ledger contains every mismatch, not only a preferred witness.

**Not evaluated.** A bearer, support, row alignment, dependency, evaluator, transport, bridge, or equality procedure is missing or unadmitted.

**Refused.** The input is malformed, noncanonical, unsupported, unsafe, or outside a declared resource policy.

**Inconsistent.** Admitted evidence has no live model, or independent certificate relations cannot jointly hold.

A positive ledger is complete: grouping records every signature and block; factorization records every fibre and factor row; comparison records every matched row; planning records every visited Bellman state. A negative ledger is equally complete. A short human witness is a projection from that ledger, never a replacement for it.

## 5 The operational language

The normative calculus has twenty-two public rules. Their names are stable even when a visual presentation abbreviates them. The current course parser and engine implement only a restricted carrier: the parser reads a simple course header followed by line-oriented JSON payloads, and the engine does not yet execute the full FORM/READ/MOVE/TEST/JOIN/PASTE proof theory. Several unhandled rules are recorded as discharged-premise events and terminal close payloads can be authored. The table is therefore the language specification, not a claim that all twenty-two rules are operationally checked end to end.

| Rule       | Name                             | Function                                                                                    |
|------------|----------------------------------|---------------------------------------------------------------------------------------------|
| QC-R01     | FORM                             | Freeze and type the complete prospective frame, or return the complete formation product.   |
| QC-R02     | READ                             | Evaluate one source or target row at its own admitted bearer.                               |
| QC-R03     | MOVE                             | Apply the total typed answer transport to a source answer.                                  |
| QC-R04     | TEST                             | Apply target-type equality and record every unequal primitive field.                        |
| QC-R05     | CLOSE-EQ                         | Close a complete admitted equality ledger as conservative.                                  |
| QC-R06     | CLOSE-NEQ                        | Close a complete nonempty mismatch ledger as nonconservative.                               |
| QC-R07     | OPEN                             | Return the order-independent blocker product without an equality verdict.                   |
| QC-R08     | PUSH                             | Push a complete finite law through a registered map by fibre summation.                     |
| QC-R09–R11 | JOIN, JOIN-EQ, JOIN-HIT          | Form and reason about a registered product query on a joint bearer.                         |
| QC-R12–R13 | PASTE, PASTE-EQ                  | Compose frames only across a typed shared boundary and recompute the outer frame.           |
| QC-R14–R18 | DOWN, LIFT, TRACE, READOUT, CARD | Derived movement, witness, serial, and theorem-card operations under explicit certificates. |
| QC-R19     | BUILD                            | Invoke one registered constructor.                                                          |
| QC-R20     | REQUEST                          | Invoke exactly one registered request.                                                      |
| QC-R21     | STATUS                           | Return the external scientific-status vector.                                               |
| QC-R22     | RETURN                           | Return the typed value, certificate, ledger, provenance, and ceiling.                       |

The thirteen normative BUILD operators are:

|            |             |                |                |
|------------|-------------|----------------|----------------|
| QC-EMBED   | QC-QUOTIENT | QC-FACTOR      | QC-WEAKEN      |
| QC-REDUCT  | QC-MUTUAL   | QC-RESPONSE    | QC-EVBASE      |
| QC-CODE    | QC-SUPPORT  | QC-REALIZATION | QC-OBSERVATION |
| QC-PROJECT |             |                |                |

They construct typed objects; they do not silently license a preservation conclusion.

The twenty-two registered REQUEST operators are listed in [Appendix A](#). One course contains exactly one request so that its certificate has one unambiguous conclusion. A later course may consume a prior certified object by fingerprint.

The sole STATUS operator, QC-SCIENCE, returns

$$S_C = (\text{roots, gauge, anchors, fit, freeze, reader, source, empirics}).$$

The coordinates retain separate provenance. The status vector cannot be inserted into a grammar or used to upgrade a formal equality to empirical adequacy.

## 6 Grouping, kernels, and quotient carriers

Let  $H$  be a finite carrier,  $U$  a registered finite question universe, and  $B \subseteq U$  a battery. Define

$$h \sim_B h' \iff \forall q \in B \ q(h) = q(h'). \quad (2)$$

Reflexivity, symmetry, and transitivity follow coordinatewise, so  $\sim_B$  is an equivalence relation. QC-GROUP returns its partition, signatures, and quotient map. With  $B = \emptyset$  and nonempty  $H$ , all objects share the empty signature and form one block.

The direct carrier for a consumer battery is the quotient  $H/\sim_B$ . It is minimum in the exact sense that any reduction preserving every consumer in  $B$  must distinguish at least the blocks of  $\sim_B$ . This is a consumer-relative statement. Adding another consumer or a later context can refine the required carrier.

For  $h, h' \in H$ , define the discernibility edge

$$E_{h,h'} = \{q \in U : q(h) \neq q(h')\}. \quad (3)$$

Only pairs separated by the target universe matter. A battery preserves the full target kernel if and only if it hits every such edge. Duplicate edges and any edge strictly containing another may be deleted without changing the minimal hitting sets.

### 6.1 Minimal transversal enumeration

The production recurrence begins with  $\mathcal{T}_0 = \{\emptyset\}$ . For each reduced edge  $E_{i+1}$  it sets

$$\mathcal{T}_{i+1} = \min_{\subseteq}(\{T \in \mathcal{T}_i : T \cap E_{i+1} \neq \emptyset\} \cup \{T \cup \{q\} : T \in \mathcal{T}_i, T \cap E_{i+1} = \emptyset, q \in E_{i+1}\}). \quad (4)$$

The  $\min_{\subseteq}$  operator removes every strict superset of another candidate. The final antichain is the complete family of inclusion-minimal batteries. Minimum cardinality and minimum exact cost are optimizations over that family; neither replaces inclusion minimality.

**Proposition 6.1** (Soundness and completeness of (4)). *After processing  $i$  edges,  $\mathcal{T}_i$  is exactly the family of inclusion-minimal transversals of those  $i$  edges.*

*Proof.* Induct on  $i$ . The empty set is the unique minimal transversal of the empty edge family. At step  $i + 1$ , an old transversal that already hits  $E_{i+1}$  remains a transversal. Every old transversal that misses the edge must acquire at least one member of  $E_{i+1}$ , and the recurrence constructs all such one-element extensions. Conversely, every new transversal contains one of these retained or extended candidates. Removing strict supersets cannot delete a minimal transversal and removes every nonminimal candidate. Thus soundness, completeness, and inclusion minimality are preserved.  $\square$

An empty reduced edge proves impossibility: two target-distinct objects agree on every registered diagnostic. It is a complete negative certificate, not a search failure.

## 7 Selected fibres, evidence cores, and decisions

For selected  $h^*$ , the full-universe fibre is

$$F_U(h^*) = \{h \in H : \forall q \in U, q(h) = q(h^*)\}.$$

A selected-fibre battery need only separate  $h^*$  from every outsider. Its edges are  $E_{h^*,h}$  for  $h \notin F_U(h^*)$ . It need not distinguish two outsiders and can therefore be smaller than a kernel basis.



For exact evidence  $E$ , define the live set

$$\text{Live}(E) = \{h \in H : h \text{ agrees with every observation in } E\}.$$

An evidence core is an inclusion-minimal  $E' \subseteq E$  such that  $\text{Live}(E') = \text{Live}(E)$ . Evidence is supplied fact; a selected-fibre basis is a set of possible questions. Even when their question names coincide, their result sorts and certificates remain different.

They coincide under an exact restricted condition: the evidence must be the injective image of one selected object's answers to the same battery, it must be consistent, and the complete evidence live set must equal the selected full-universe fibre. This is the content of QC-T07.

Let  $D : H \rightarrow Y$  be an independently registered decision. Battery  $B$  determines  $D$  precisely when

$$\ker(q_B) \subseteq \ker(D). \quad (5)$$

Equivalently,  $B$  hits every edge  $E_{h,h'}$  for which  $D(h) \neq D(h')$ . The target decision must not be added to the diagnostic registry; doing so would make the singleton target a vacuous predictor.

*Proof of the decision hitting equivalence.* If (5) holds and a target-disagreeing pair had no chosen separator, the pair would agree on  $B$ , contradicting factorization. In the other direction, if a pair agrees on  $B$  but has different decisions, its decision edge must be hit by some  $q \in B$ , contradicting agreement. The registered universe is required to factor the decision so every edge is expressible inside that universe.  $\square$

Kernel bases, selected-fibre bases, evidence cores, and decision factorizations therefore have four nominal result types. There is no general upcast among them.

## 8 Query price and the information lattice

Let  $b : X \rightarrow A$  encode the current battery signature and  $q : X \rightarrow B$  the added query. The global added-consumer price is

$$\text{Price}(q \mid b) = |\text{im}(b, q)| - |\text{im}(b)|. \quad (6)$$

For each old block  $C_a = b^{-1}(a)$ , let  $k_a = |q(C_a)|$  be the number of realized added labels inside that block. Then

$$\text{Price}(q \mid b) = \sum_{a \in \text{im } b} (k_a - 1). \quad (7)$$

*Proof.* The reachable joint image is in bijection with the disjoint union of the realized  $q$ -images inside each old fibre:

$$\text{im}(b, q) \cong \coprod_{a \in \text{im } b} q(C_a).$$

Hence  $|\text{im}(b, q)| = \sum_a k_a$ . Every old block is nonempty, so  $k_a \geq 1$ . Subtracting  $|\text{im } b| = \sum_a 1$  gives (7). Price is zero exactly when every  $k_a = 1$ , which is equivalent to  $q$  being constant on every  $b$ -fibre, or  $\ker(b) \subseteq \ker(q)$ .  $\square$

Adding a question already present in the canonical battery is therefore an evaluated zero-price operation. The result also records `already_present`; zero is not confused with an omitted calculation.

The information lattice quotients syntactically different batteries by their induced partition. Order is partition refinement. Redundancy closure adds every registered question constant on the current kernel. It is extensive, monotone, and idempotent. Closed batteries are fixed points; their meet is intersection and their join is closure of union. Hasse edges represent partition covers rather than necessarily one-question edits.

## 9 Contextual adequacy

A direct carrier preserves answers already read. A contextual carrier must preserve answers after every continuation generated by a declared finite signature of typed partial operations.

An equivalence  $\equiv \subseteq \equiv_0$  is a strong congruence for a partial  $n$ -ary operation  $f$  when equivalent arguments agree on definedness and, when defined, yield equivalent results:

$$\vec{x} \equiv \vec{y} \implies (f(\vec{x}) \downarrow \iff f(\vec{y}) \downarrow),$$

$$\vec{x} \equiv \vec{y} \wedge f(\vec{x}) \downarrow \implies f(\vec{x}) \equiv f(\vec{y}).$$

Definedness is semantic. If an operation applies to one representative but not another, merging them conceals a possible future contrast.

The finite algorithm starts from the direct partition and repeatedly splits blocks by the definedness and successor-block signatures of every registered operation context. Every nontrivial pass strictly refines a finite partition, so the algorithm terminates. At the fixed point, equal states have equal context signatures and hence form a strong congruence.

**Theorem 9.1** (Greatest contextual congruence). *The fixed-point relation  $\equiv_{\text{ctx}}$  is the greatest strong congruence contained in the direct observation kernel  $\equiv_0$ .*

*Proof.* The splitting invariant keeps every block inside one  $\equiv_0$  block. Fixed-point equality of all one-hole context signatures gives definedness and successor compatibility, so the result is strong. Let  $\rho$  be any other strong congruence below  $\equiv_0$ . Initially every  $\rho$ -pair lies in one direct block. If it lies together before a refinement pass, strength of  $\rho$  makes the pair agree on definedness and places corresponding successors in the same pre-pass block. The pass cannot split that pair. Induction over the finite passes gives  $\rho \subseteq \equiv_{\text{ctx}}$ .  $\square$

Maximality is relative to the admitted signature. A later process outside that signature requires a new record and fingerprint.

## 10 Recovery and first irreversible loss

Semantic-orbit recovery and raw-representative recovery answer different questions. Let  $r : X \rightarrow Y$  be a reduction respecting registered source and target equivalence relations. It induces an orbit map  $\bar{r} : X/\sim_X \rightarrow Y/\sim_Y$ . On a nonempty finite source orbit space,  $\bar{r}$  has a two-sided inverse on its range exactly when it has no orbit collisions. Raw noninjectivity can therefore coexist with semantic recovery if collapsed representatives lie in one semantic orbit.

Raw recovery is stronger. It requires injectivity on the requested image, stability of the target image under every requested action, total lifted actions, identity and composition preservation, and the commuting condition

$$r(\tilde{g}(x)) = g(r(x)).$$

Each failed conjunct receives its own negative witness.

For a pipeline

$$X_0 \xrightarrow{r_1} X_1 \xrightarrow{r_2} \dots \xrightarrow{r_n} X_n$$

and fixed consumer  $Q : X_0 \rightarrow A$ , write  $R_i = r_i \circ \dots \circ r_1$ . Stage  $i$  preserves the consumer exactly when  $\ker(R_i) \subseteq \ker(Q)$ . If loss occurs, the finite ordered index set has a least failing stage. The certificate supplies a  $Q$ -disagreeing  $R_i$ -collision there and positive factor maps at every earlier stage.

No downstream deterministic suffix receiving only  $R_i(x)$  can reconstruct a lost  $Q$  distinction: if  $R_i(x) = R_i(y)$ , then  $s(R_i(x)) = s(R_i(y))$  for every suffix  $s$ . A side channel changes the carrier and must be registered rather than invoked informally.

## 11 Transports, bridges, and comparison

The stable transport registry names `IDENTITY`, `RENAMING_BIJECTION`, `FINITE_TOTAL_MAP`, `TUPLE_PROJECTION`, `FINITE_MERGER`, `PROBABILITY_PUSHFORWARD`, `RELATION_TRANSPORT`, `CONTEXT_HOMOMORPHISM`, and `SERIAL_READOUT`. The normative contract requires every transport to be type-correct and total on its licensed source image. The current Python constructor enforces total mapping and the bijection condition for a renaming bijection, but most other kind names do not yet dispatch to kind-specific semantic checks. A kind label must therefore not be treated as a verified merger, pushforward, relation transport, context homomorphism, or serial readout.

For a finite map  $f : X \rightarrow Y$  and exact mass  $P : X \rightarrow \mathbb{Q}$ , pushforward is

$$(f_*P)(y) = \sum_{x:f(x)=y} P(x).$$

It preserves total mass:

*Proof.* Because  $f$  is total, the fibres  $f^{-1}(y)$  partition  $X$ . Therefore

$$\sum_{y \in Y} (f_*P)(y) = \sum_{y \in Y} \sum_{x \in X} [f(x) = y] P(x) = \sum_{x \in X} P(x).$$

For each  $x$ , exactly one indicator is one. The equality holds over exact rationals; no numerical tolerance is involved.  $\square$

A normative bridge crosses a domain or scientific layer and carries its assumptions, source evidence, total carrier map, answer transport, and commutative-square certificate. A failed square should produce `QC-A005`, not an approximate comparison. In the current public comparison path, cross-domain or cross-layer inputs block; a separately defined bridge object is not yet consumed to rescue that comparison. Bridge-aware comparison therefore remains an integration obligation. The current same-domain support gate checks equal support cardinality; it does not by itself verify a scientifically correct row correspondence. That alignment must be declared and audited rather than inferred from equal lengths.

Once formation and admission pass, the comparison trichotomy follows by case analysis on exact equality. If either gate fails, equality is not called and the outcome is `NOT_EVALUATED`. This is `QC-T16` in the Lean projection and in the executable comparison contract.

## 12 Serial semantics

For a finite transition system with initial state  $s_0$ , stopping set  $F$ , and readout  $\rho$ , the stopped language is

$$L_{\text{stop}} = \{\rho(\pi) : \pi \text{ begins at } s_0 \text{ and first stops in } F\}.$$

Structural nontermination records a reachable path that can avoid  $F$ . Stochastic nontermination records its exact mass; a structurally reachable zero-probability cycle contributes zero mass.

For rational transition probabilities with transient matrix  $Q$  and absorbing matrix  $R$ , the exact absorbing law is

$$B = (I - Q)^{-1}R$$

when the registered conditions hold. The stopped masses and nontermination mass sum to one. Weighted path series sum the weights of all paths bearing one readout instead of deduplicating them.

Nominal separation prevents a terminal-output comparison from silently becoming a trajectory comparison. SERIAL\_READOUT is the only intentionally forgetful serial transport and must state what information it discards.

### 13 Finite inference and experiment design

The inference layer uses a closed, many-sorted, bounded formula language over an admitted finite model class. Unbounded quantification and arbitrary host callbacks are refused.

For live set  $L$  and evidence formula  $e$ , UPDATE is exact intersection:

$$\text{Update}(L, e) = \{h \in L : h \models e\}.$$

Membership immediately gives

$$h \in \text{Update}(L, e) \iff h \in L \wedge h \models e,$$

so no new model is introduced. Reapplying the same exact evidence is idempotent; finite intersections make consistent evidence order independent.

ENTAIL returns ENTAILED when all live models satisfy the formula, REFUTED when all satisfy its negation, UNDERDETERMINED when both truth values occur, and INCONSISTENT when the live set is empty. The last branch prevents empty-set vacuity from licensing every conclusion.

The current DISCOVER implementation enumerates atomic EQ and NEQ facts over registered question values within declared candidate bounds. The wider specification admits a bounded well-typed formula language, but the present implementation is not a general higher-order or full-formula synthesizer. For original model class  $M$ , formula  $\varphi$  has truth vector

$$v_M(\varphi) = (\mathbf{1}[m \models \varphi])_{m \in M}.$$

Discovery quotients formulas by this vector, retains a least-complex canonical representative, removes tautologies, contradictions, evidence, and axioms, and returns only consequences forced by the current live set. Completeness is therefore explicitly relative to the bounded syntax and truth-vector quotient.

For a candidate diagnostic  $q$ , let  $L_{q=a} = \{h \in L : q(h) = a\}$ . ASK minimizes lexicographically

$$\left( \max_a |D(L_{q=a})|, \max_a |L_{q=a}|, \text{cost}(q) \right),$$

retaining every tie before choosing a canonical display identifier.

PLAN extends the criterion to finite horizon. In the scalar formal projection its Bellman recurrence is

$$V_0(s) = T(s), \quad V_{d+1}(s) = \min_a (c(s, a) + V_d(\delta(s, a))).$$

The operational planner uses the registered lexicographic worst-response tuple and retains every tied optimal root. The certificate contains every visited state value, so a checker can verify the recurrence without trusting the optimizer's search order.

### 14 Grammar adapters and ranking spaces

The repository contains adapter modules intended to preserve the native semantics of each formalism rather than forcing every grammar into a single answer shape. Their status is uneven. The general

executable-evaluator registry is empty by default in the inspected checkout and no production registration call connects these adapters to that dispatcher; sealed extensional tables remain the dependable general-record route. The descriptions below state the mathematical contract and identify implemented specialized functions without treating module presence as integrated dispatch.

Strict OT accepts labelled candidates, multiplicity, nonnegative integer violation vectors, total rankings, tied strata, and finite partial-order precedence. At stratum sequence  $S_1, \dots, S_k$ , candidate  $c$  defeats  $d$  at the first stratum with a smaller sum:

$$c \prec d \iff \exists j \left[ \forall i < j, \sum_{r \in S_i} v_r(c) = \sum_{r \in S_i} v_r(d) \wedge \sum_{r \in S_j} v_r(c) < \sum_{r \in S_j} v_r(d) \right].$$

A tie is a selected-candidate set, not an exception erased by the engine.

The finite-HG adapter module uses exact rational weights and integer violations:

$$H_w(c) = \sum_i w_i v_i(c),$$

retaining all tied minima. The MaxEnt specification uses positive base mass  $\mu(c)$  and input-specific normalizer

$$P(c \mid x) = \frac{\mu(c) e^{-w \cdot v(c)}}{Z_x(w)}, \quad Z_x(w) = \sum_{d \in C_x} \mu(d) e^{-w \cdot v(d)}.$$

Across inputs, unrelated normalizers cannot be cancelled. Labelled duplicate rows remain distinct unless a registered mass-preserving fusion transports their full fibre. The current MaxEnt adapter constructs exact same-input and cross-input relative-partition ledgers; it does not expose a general routine that evaluates normalized probabilities from weights. Such a ledger is not a probability engine result.

The continuous-HG module exposes exact special cases and theorem-shaped data for the declared objective

$$H_{N,p}(x) = h \sum_{i=1}^N (x_{i-1} - x_i)_+^p + m \sum_{i=1}^N x_i, \quad x_0 = 1, \quad x_i \in [0, 1].$$

For  $p > 1$ , it uses exact algebraic comparisons for

$$q = \frac{1}{p-1}, \quad a = \left( \frac{m}{ph} \right)^q, \quad K_p = \min \left\{ k : a \sum_{r=1}^k r^q \geq 1 \right\}.$$

The quadratic path is computed exactly. For general  $p > 1$ , the current `solve` function returns a symbolic placeholder carrying parameters and the first-zero threshold rather than a constructed exact optimizer. The finite operational classifier does not infer a continuum theorem from that placeholder or from a displayed numerical grid.

## 14.1 Query-relative ranking-space compression

For a finite partial order on constraints, the ranking-space engine counts every compatible total ranking without materializing the factorial list. Its state is the ranked constraint set  $M$  and the query-relevant semantic state  $s$ . With admissible next constraints  $A(M)$ ,

$$F(M, s) = \sum_{c \in A(M)} F(M \cup \{c\}, \text{step}_c(s)).$$

Equal states merge. If every surviving candidate has the same value on every remaining constraint, processing the suffix leaves the survivor set unchanged.

**Lemma 14.1** (Constant-suffix lemma). *Let  $L$  be the current survivor set. If for every remaining constraint  $c$  there is a value  $k_c$  with  $v_c(x) = k_c$  for all  $x \in L$ , then every ordering of the remaining constraints retains exactly  $L$ .*

*Proof.* One constant constraint assigns the same value to every survivor, so its minimum-retention step is the identity on  $L$ . Induct over the remaining sequence. Each step is the identity, hence their composition is the identity.  $\square$

The suffix then contributes its exact linear-extension count to the current answer class. Shares are reduced rational counts of compatible rankings. They are not token probabilities unless a probability measure and response law are independently declared.

## 15 Using Q-Calculus

### 15.1 Installation and local command line

The standalone reference package requires Python 3.12 and uses the locked project environment. From its repository root:

```
make bootstrap
source .venv/bin/activate
qcalc analyze fixtures/workbench/chapter1_winner.toml
```

The compact TOML workbench supports four document kinds:

```
ranking_space           ranking_comparison
constraint_clone_audit   finite_study
```

Integers and quoted rationals are accepted on exact paths; TOML floating-point values are refused.

### 15.2 A ranking-space study

```
kind = "ranking_space"
title = "One winner, two complete orders"
query = "winner"
constraints = ["C1", "C2"]
```

```
[[tableau]]
id = "input-x"
```

```
[[tableau.candidate]]
id = "a"
violations = [0, 0]
```

```
[[tableau.candidate]]
id = "b"
violations = [0, 1]
```

```
[[tableau.candidate]]
id = "c"
violations = [1, 0]
```

Run it with

```
qcalc analyze study.toml
qcalc analyze --json study.toml
```

Changing the query from `winner` to `order` can reveal two answer classes even when the winner is invariant. This is a difference in the declared answer type, not a contradiction.

Optional precedence rows are pairs `[higher, lower]`. Candidate multiplicity is a positive integer. The optional candidate field named `output` in the workbench schema is an overt readout label used by `overt_winner`; the row is still a candidate until evaluation selects it.

### 15.3 Comparing aligned ranking spaces

Use `kind = "ranking_comparison"` with `source_tableau` and `target_tableau` arrays. Both sides are evaluated under each same compatible ranking. An optional `[transport]` table maps source answer labels to target labels. The result reports exact matching and mismatching ranking counts, every aligned answer class, and a witness ranking for each class.

### 15.4 A finite cross-formalism study

```
kind = "finite_study"
title = "Diagnostic design"
target = "analysis"
diagnostics = ["surface", "derivation"]
horizon = 1

[observed]
surface = "A"

[costs]
surface = 1
derivation = "3/2"

[[model]]
id = "strict-ot"
formalism = "Strict OT"
[model.answers]
surface = "A"
derivation = "short"
analysis = "categorical-parallel"
```

Add further `[[model]]` tables to define the complete admitted model class. The result gives the live models, model-derived invariants, minimum target batteries, collision ledger when the target is not identifiable, ASK, and a bounded plan. A row labelled HG or MaxEnt is a declared prediction unless its evaluator and provenance are separately admitted.

### 15.5 Full JSON protocol

Use the full protocol for the currently implemented record and request operations, theorem-card calls, identity checks, or service integration:

```
qcalc schema
qcalc validate analysis.json
qcalc run analysis.json
```

```
qcalc group group-request.json
qcalc basis basis-request.json
qcalc compare comparison-request.json
qcalc verify-certificate certificate-check.json
```

Standard input is selected with `-`. Protocol commands emit canonical JSON on standard output and diagnostics on standard error. Evaluated positive and negative verdicts exit zero; a refusal exits two; not evaluated exits three; inconsistent evidence exits four; and certificate failure exits six. The current run course surface is not a complete interpreter for the twenty-two-rule normative grammar, bridge execution is not integrated into public comparison, and `verify-certificate` checks fingerprint identity rather than semantic validity. Prefer the specific request commands whose implementation and verifier have been audited for the intended task.

## 15.6 API and PhonoScript surfaces

The stateless service publishes health, capability, registry, and schema GET routes plus POST routes for validation, course execution, grouping, bases, factorization, price, ASK, PLAN, UPDATE, ENTAIL, DISCOVER, comparison, contextual carriers, recovery, first loss, theorem cards, and certificate verification. The OpenAPI document and TypeScript SDK preserve exact tagged integers and rationals.

PhonoScript exposes the integrated finite ranking operations

```
q_ranking_space() -> record
q_clone(constraint) -> record
typology() -> record
```

on the current `.ottab` project. In that surface the current engine requires strict OT, the retain-all tie policy, aligned nonempty tableaux, and the documented finite state budget. It returns a structured refusal rather than an approximation when those conditions fail.

## 16 Certificates, canonical identity, and trust

Canonical JSON is UTF-8 normalized to NFC, has keys sorted by Unicode code point, contains no insignificant whitespace, and uses tagged exact numbers. Mathematical sets sort by canonical element bytes; declared sequences retain order. Duplicate canonical keys and normalization collisions are refused.

Every result envelope records protocol and package versions, operation, execution status, verdict, record and request fingerprints, algorithm identity, typed value, certificate, complete witness ledger, dependency ledger, provenance, claim ceiling, and result-dependent fingerprints.

A certificate is specified as a semantic object rather than a log. The required correctness pattern is

$$\text{Verify}(R, P, v, c) = \text{true} \implies v = \text{Sem}(R, P) \quad (8)$$

at the registered scope. A matching hash proves byte identity only. A semantic verifier must also recompute the mathematical conditions. This distinction exposes a current implementation gap: the public `verify-certificate` service in the inspected checkout recomputes and compares the certificate fingerprint, but does not dispatch to the operation-specific semantic verifiers. It therefore establishes identity, not Equation (8), and must not be cited as an independent semantic replay.

The same distinction applies to provenance and dependencies. A well-shaped authored provenance status is not a universally computed, non-forgable scientific status, and a digest-shaped dependency string does not prove the authenticity or correctness of its source. Source admission and scientific review remain separate obligations.



The intended trust architecture is layered:

- the operational implementation decodes closed records, computes finite semantics, and constructs certificates;
- operation-specific verifier modules should recompute certificate conditions and compare optimized algorithms with structurally simpler bounded oracles;
- Lean checks theorem declarations and finite proof projections under explicit assumptions;
- Wolfram is intended to replay applicable exact symbolic and algebraic fixtures without importing the operational answer; and
- source admission and claim ceilings bind formal objects to their bounded scientific interpretation.

The current formal tree contains the twenty-six declarations catalogued below and a coverage mapping for sixty-eight dissertation result identifiers and 218 proof-goal identifiers. The checked-in Lean build log records a successful compilation. At the same time, the theorem registry marks all twenty-six rows `PENDING_PROOF`, the proof-goal coverage rows await the final axiom audit, the sixty-eight result rows are source-admitted rather than release promoted, and the current weakness and mutation registries remain open or pending. Those counts are therefore registration and proof-surface facts, not a completed release certificate.

## 17 Proof ledger for QC-T01 through QC-T26

This section states the operational theorem spine and gives the proof used by the finite mathematical projection. Several results are definitions or finite equivalences. Stronger production behavior has targeted verifier modules, unit/property tests, and independent bounded oracles, but the public certificate endpoint does not yet dispatch those semantic verifiers and the registry status remains pending.

### 17.1 Formation and structural theorems

**QC-T01 (deterministic formation).** A fixed raw finite record has at most one formed result. If `form-Record(raw)` equals both `some(first)` and `some(second)`, rewriting the second equality with the first gives `some(first) = some(second)`; injectivity of the `some` constructor yields equality. Complete blocker products and hash-seed independence belong to the operational formation verifier.

**QC-T02 (canonical round trip).** The formal finite encoding places the version, field count, and field sequence in order. Decoding checks nonnegativity and exact length. Substitution of the encoded list makes every check reflexive, so `decode after encode` is the original record. The protocol extends this pattern with type-specific canonicalizers and fingerprint tests.

**QC-T03 (GROUP correctness).** `GROUP` is defined as the setoid whose relation is equality on every battery coordinate. The theorem is therefore definitional equality with (2). The executable certificate additionally materializes all blocks, signatures, and pair checks.

**QC-T04 (discernibility hitting).** Suppose the chosen kernel refines the full kernel. If a full-distinct pair had no chosen question in its edge, it would agree on every chosen coordinate, contradicting refinement. Conversely, if every full-distinct pair edge is hit, a pair agreeing on all chosen coordinates cannot be full-distinct, because the hitting coordinate would both agree and disagree. The assumption  $B \subseteq U$  is explicit.

**QC-T05 (minimal transversals).** The Lean certificate projection defines the reference family by filtering the finite powerset for edge coverage and absence of a smaller covering subset; membership is consequently equivalent to domain inclusion and minimal-transversal status. The production Berge antichain algorithm is justified by the induction following (4) and compared with this powerset oracle on bounded instances.

**QC-T06 (selected-fibre hitting).** Repeat the QC-T04 argument with one endpoint fixed at  $h^*$  and quantify only over outsiders to  $F_U(h^*)$ . A missed outsider edge creates a chosen-kernel collision; a hit contradicts any such collision.

**QC-T07 (selected fibre and evidence).** Map each chosen question  $q$  injectively to observation  $(q, q(h^*))$ . An object satisfies the resulting evidence exactly when it agrees with  $h^*$  on the chosen battery. Replacing the chosen battery with  $U$  gives the full fibre. Hence preservation of the selected fibre is equivalent to preservation of the two live sets under this specific image construction. The result constructors remain distinct by nominal typing.

**QC-T08 (decision factorization).** This is the decision-edge proof given after (5). It requires that the full registered universe already factors the decision.

**QC-T09 (nominal result separation).** The four result constructors `kernelBasis`, `selectedFibreBasis`, `evidenceCore`, and `decisionFactorization` are distinct members of an inductive type. Constructor disjointness proves that these four tags cannot be equal. The Lean declaration does not by itself prove every semantic nonimplication among the mathematical notions; those require the separate finite counterexamples and their assumptions.

**QC-T10 (exact price).** Equations (6) and (7) and their fibre-bijection proof establish blockwise price equality. Zero price is equivalent to query redundancy.

**QC-T11 (closure and lattice).** Redundancy closure contains the input battery, grows monotonically under battery inclusion, and is idempotent because closing twice adds no question beyond those already constant on the same kernel. Intersections of fixed points are fixed; closure of a union is the least fixed point above both. These are the meet and join laws for the closed battery lattice.

## 17.2 Context, recovery, and comparison theorems

**QC-T12 (greatest contextual congruence).** The fixed-point proof is given in Section 8. The Lean theorem is stated for arbitrary finitary many-sorted signatures and imports the complete one-hole-context construction.

**QC-T13 (semantic-orbit recovery).** The registered weakening respects source and target setoids and therefore induces a map on quotient orbits. For a finite nonempty source quotient, an empty collision set is injectivity. An injective map has an inverse on its range by choosing the unique preimage; the inverse laws are immediate. Conversely, a two-sided inverse on the range implies injectivity and hence no collisions.

**QC-T14 (raw recovery).** The formal declaration proves the registered equivalence among representative injectivity, stability of the requested target image, and commutation of the supplied action lifts with reduction. The executable negative result records witnesses for failed registered conditions. Identity and composition laws for an action family are stronger coherence obligations discussed by the specification; they are not conjuncts proved by this Lean declaration.

**QC-T15 (least first loss).** The failing stages form a nonempty finite subset of a linearly ordered index type. Its minimum exists. By membership it fails factorization; by minimality every earlier stage factors the consumer. This is the theorem's formal scope. The operational certificate additionally aims to attach a collision at the minimum and factor maps before it. The Lean declaration does not establish the no-loss branch, pipeline composition, or persistence of loss under every later transformation.

**QC-T16 (finite comparison trichotomy).** Case-split on the two Boolean gates formed and admitted. If either is false, the result is not evaluated. If both are true, decidable equality of the answer type gives exactly conservative or nonconservative. The cases are exhaustive and pairwise disjoint.

**QC-T17 (finite transport sum identity).** A total finite map produces a target for every source item, and reindexing its rational weights by target fibres preserves the total finite sum. This is an algebraic pushfor-

ward identity. Nonnegativity, normalization to one, divergence policy, and the semantics of each named transport kind are additional formation or admission conditions, not conclusions of this declaration.

### 17.3 Inference, planning, and verification theorems

**QC-T18 (UPDATE correctness).** UPDATE is finite filtering. Membership in a filter is equivalent to membership in the original state and truth of the formula, so the new set is a subset and contains exactly the admitted models.

**QC-T19 (ENTAIL correctness).** In the formal projection, entailment is defined as truth for every member of the finite state. Soundness and completeness relative to that semantics are definitional. The operational result separately handles the empty state as inconsistent.

**QC-T20 (bounded DISCOVER correctness).** The finite reference constructor maps every bounded formula code to its truth vector. Membership in the image is equivalent to existence of a candidate code with that vector. This proves finiteness of the truth-vector image. Nontriviality, canonical representative selection, removal policies, minimal supports, and operational resource limits lie outside that Lean statement and require separate implementation checks.

**QC-T21 (ASK optimality).** ASK filters the finite experiment set by the predicate that its score is no greater than every admitted rival's score. Finite-set filter membership is exactly admissibility plus that universal minimum condition. A lexicographic score type supplies the registered minimax ordering, and all equal minima remain in the filter.

**QC-T22 (PLAN optimality).** The formal value function is defined by primitive recursion on the horizon. Unfolding the successor case is exactly the defining Bellman equation. The declaration does not prove that a concrete returned plan tree contains every licensed branch, retains every tied optimum, or is optimal for an independently defined operational planner. Those are separate verifier and test obligations.

**QC-T23 (serial no-cast).** Each serial payload has a constructor tag. Equality of two payloads implies equality of their tags by congruence. Since the tags are distinct constructors, differently tagged payloads cannot be equal. This nominal result does not make an explicitly authored forgetful cast impossible; the surrounding formation rules must require an admitted SERIAL\_READOUT for such a comparison.

**QC-T24 (proof-carrying response import).** A response-import record contains four proof fields: reader after encoding equals the source semantic query; its equality Boolean is exact; descent commutes with the semantic transformation; and every mismatch code has a source preimage. Conjunction of those fields yields recovery, equality preservation, transformation compatibility, and mismatch witnesses at the declared finite scope.

**QC-T25 (verifier soundness).** The formal Boolean certificate verifier recomputes the conjunction of its input and compares it with the claim. Acceptance means Boolean equality with that recomputation. This theorem is a minimal sound-verifier pattern, not proof that the current public `verify-certificate` endpoint performs semantic replay. That endpoint currently verifies fingerprint identity only; operation-specific dispatch and promoted mutation evidence remain completion obligations.

**QC-T26 (evaluator independence).** In the formal architecture, the exhibited source-expression function has no target argument. Pairing its value with either of two target payloads therefore leaves the first projection definitionally unchanged. This witnesses the intended dependency separation; it is not a universal noninterference theorem for every implementation path. The corresponding semantic mutation remains listed as pending in the current mutation manifest.

## 18 Reproducibility, performance, and security

The reproducibility specification pins the Python, Lean, Mathlib, Wolfram, and Node boundaries recorded by the project. Narrow development checks do not substitute for the full orchestrated release workflow, and the absence of its final reports in the inspected checkout means this manual does not assert that the release workflow has completed.

The checked-in Python report records 1,295 passing executions (259 cases under each of five invocations, comprising four deterministic hash-seed runs and one coverage run), zero failures and skips, 98.89 percent overall statement coverage, 98.57 percent overall branch coverage, and 100 percent coverage for the designated semantic core. The exhaustive report records 6,116,139 bounded cases with zero mismatches. The checked-in Rust report also records 35 passing Studio tests. Its fixed ranking fixture represents 20,922,789,888,000 compatible rankings, five answer classes, and 17 dynamic states in 18,239 microseconds; it also records 50,000 first-order evaluations over 4.8 million cells in 92,436 microseconds and 20,000 typed comparisons in 97,013 microseconds. These figures describe the checked-in reports, not a newly rerun release in this manual build. The repository also specifies deterministic regeneration, exhaustive small finite universes, property laws, certificate mutation rejection, application replays, and cross-engine cases. Sixty-two Wolfram test definitions are present, but the corresponding execution report is absent. The Lean build log records a successful 17,578-job build. Because the final cross-engine, Wolfram, axiom-audit, mutation, and release-marker reports are absent, those latter items remain specified gates rather than current pass claims.

The fixed ranking benchmark has sixteen constraints, four tableaux, and six candidates per tableau. It represents

$$16! = 20,922,789,888,000$$

total rankings and compresses them by query-relevant states rather than enumerating the permutation list. Performance budgets are release gates on their recorded machines, not universal promises for arbitrary combinatorial inputs.

Public inputs are untrusted. The semantic boundary contains no intended dynamic code selection. Implemented decoders reject many floating-point, rational, canonicity, probability, dependency, transport, nesting, and size errors. Limits must be checked at the actual service boundary: for example, the API defines a 120-second timeout constant, but the inspected middleware does not enforce that timeout. Finite basis and planning problems remain combinatorial; an unenforced declared limit is documentation, not protection.

The stateless service does not supply authentication, transport encryption, multi-tenant isolation, persistence, deployment rate limiting, or host-process sandboxing. Those are deployment responsibilities outside semantic fingerprints.

## 19 Scientific use and claim ceilings

The result travels with provenance and a claim ceiling. The principal routes are FORMALLY\_PROVED, EXACTLY\_COMPUTED, CROSS\_ENGINE\_VERIFIED, MODEL\_DERIVED, SOURCE\_DERIVED, PROSPECTIVE\_PREDICTION, OBSERVED, SEALED\_EXTENSIONAL, and NOT\_EVALUATED. They are evidence routes, not one prestige scale.

Grammar, acquisition, phonetic realization, observation and annotation, listener response, and corpus inference remain distinct layers. A bridge can relate layers but cannot identify a unique upstream cause from downstream equality. Continuous measurements are compatible with grammar-internal continuity and with categorical grammar followed by continuous realization unless discriminating evidence excludes one.

For responsible reporting, record package and protocol versions, record and request fingerprints, operation, execution status, verdict, algorithm identifier, result fingerprint, provenance, and claim ceiling. For a theorem card, also record its result identifier and statement digest. For a refusal, report the complete blocker ledger: it states what must be supplied before the question becomes answerable.

## A Request reference

| Request                   | Result                                                                  |
|---------------------------|-------------------------------------------------------------------------|
| QC-SERIAL-REDUCT          | Typed serial reduction with preserved answer semantics.                 |
| QC-PROPERTY               | One registered finite property outcome.                                 |
| QC-ORBIT-RECOVERY         | Orbit recovery map or complete collision witnesses.                     |
| QC-RAW-RECOVERY           | Coherent lift family or complete failed-condition witnesses.            |
| QC-DIRECT-CARRIER         | Direct quotient carrier and quotient map.                               |
| QC-CONTEXT-CARRIER        | Greatest strong congruence below the direct kernel.                     |
| QC-FRONTIER               | Certified nondominated finite architecture frontier.                    |
| QC-ARCH                   | Architecture admission outcome and complete ledger.                     |
| QC-FIRST-LOSS             | NO_LOSS or least loss stage and collision ledger.                       |
| QC-GROUP                  | Battery-kernel partition, signatures, and quotient map.                 |
| QC-KERNEL-BASIS           | All inclusion-minimal full-kernel batteries and exact optima.           |
| QC-SELECTED-FIBRE-BASIS   | Full selected fibre and all minimal outsider-separating batteries.      |
| QC-EVIDENCE-CORE          | All minimal evidence subsets preserving the live set.                   |
| QC-DECISION-FACTORIZATION | Minimal predictor batteries, factor maps, or collision ledgers.         |
| QC-DEPENDENCY             | Dependency closure or all minimal blocker sets.                         |
| QC-PRICE                  | Exact blockwise splitting price and idempotence flag.                   |
| QC-ASK                    | Every tied one-step lexicographic minimax optimum.                      |
| QC-PLAN                   | Tied optimal roots, canonical plan tree, and dynamic-programming table. |
| QC-UPDATE                 | New live set, removals, provenance, and inconsistency cores.            |
| QC-ENTAIL                 | Entailed, refuted, underdetermined, or inconsistent with witnesses.     |
| QC-DISCOVER               | Canonical bounded consequences, supports, and counterfactuals.          |
| QC-INCONSISTENCY-CORE     | Every inclusion-minimal inconsistent evidence core.                     |

## B Refusal and blocker codes

| Codes        | Class               | Meaning                                                                                                                                                 |
|--------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| QC-F001-F004 | Formation           | Schema failure, unknown or duplicate identifier, or type mismatch.                                                                                      |
| QC-F005-F007 | Formation           | Incomplete carrier/table, incomplete support, or unformed evaluator.                                                                                    |
| QC-F008-F012 | Formation/admission | Dependency cycle or absence, missing domain/layer bridge, or partial transport.                                                                         |
| QC-F013-F016 | Formation           | Inexact value on an exact path, unregistered cast, resource excess, or noncanonical value.                                                              |
| QC-F017-F024 | Formation           | Incomplete target, closed-range error, illegal course, changed prospective target, unsupported operation, invalid context, ranking, or stopping policy. |
| QC-A001-A003 | Admission           | Source, row alignment, or support correspondence is unavailable.                                                                                        |
| QC-A004-A007 | Admission           | Authority digest, bridge square, evaluator assumptions, or target admission failed.                                                                     |

| Codes        | Class       | Meaning                                                                                                                  |
|--------------|-------------|--------------------------------------------------------------------------------------------------------------------------|
| QC-A008-A010 | Admission   | Equality unavailable, source image uncomputable, or continuation outside the signature.                                  |
| QC-S001-S004 | Serial      | Mode mismatch, missing stopping/nontermination policy, or illegal serial cast.                                           |
| QC-C001-C005 | Certificate | Malformed or invalid certificate, engine disagreement, fingerprint mismatch, or incomplete ledger.                       |
| QC-I001-I006 | Inference   | Inconsistent evidence, evaluated nonfactorization, formula/bound failure, empty live set, or unformed response relation. |

Formation errors that make an object malformed normally produce REFUSED. Missing scientific admission conditions normally produce NOT\_EVALUATED. QC-C003, QC-I001, and QC-I005 produce INCONSISTENT. QC-I002 is an evaluated negative factorization result.

## C Paper-and-pencil procedure

For a small finite study, write carrier items down the rows and registered typed questions across the columns. To compute GROUP, circle identical battery signatures. To compute a kernel basis, form one edge for every pair separated by the full question universe, remove duplicate and dominated edges, and apply recurrence (4). To compute a selected-fibre basis, use only edges from the selected item to full-fibre outsiders.

To test decision factorization, write the decision beside each row and check whether it is constant inside every battery block. To locate first loss, form the accumulated reduction at each pipeline stage and find the least stage with a consumer-disagreeing collision.

For a source-target comparison, calculate the two sides on separate parts of the page. Apply the transport only after both reads exist. Mark every matched row and every semantic field before closing the ledger. If a required port is absent, write the blocker and no equality symbol.

The governing discipline is concise: fix the consequence, form every bearer and dependency, evaluate independently, transport through a total typed map, compare with semantic equality, and return a complete certificate or blocker ledger. Keep scientific provenance and the withdrawal condition beside the result.